

Introduction:

The objective of the project is to predict the stock price of Google, officially named as alpha. The project starts with the visualization of current data and is later presented with predicted data. In the end, the prediction will be validated with Mean absolute error and root mean squared error. In the report, the code is presented with interpretation.

```
# Import pandas to read csv file

import pandas as pd

alpha=pd.read_csv('GOOGLE.csv')
print(alpha.head())

# To get number of training days
print("trainingdays=", alpha.shape)
```

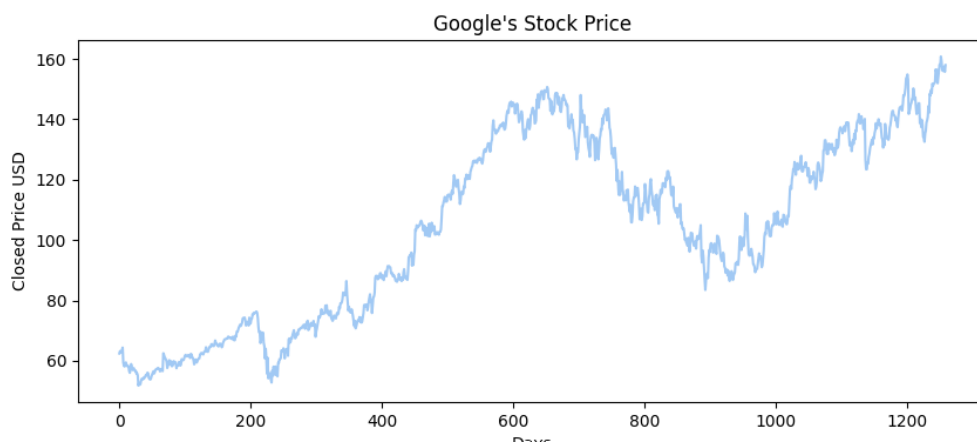
Interpretation:

	Date	Open	High	Low	Close	Adj Close	Volume
0	4/22/2019	61.799500	62.454498	61.415501	62.442001	62.442001	16146000
1	4/23/2019	62.534500	63.450001	62.319000	63.227501	63.227501	26398000
2	4/24/2019	63.206001	63.400501	62.750000	62.799999	62.799999	20376000
3	4/25/2019	63.238499	63.370399	62.601501	63.172501	63.172501	22146000
4	4/26/2019	63.450001	63.653500	63.015999	63.609001	63.609001	24828000

trainingdays= (1260, 7)

```
# To visualize close price data
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_palette('pastel')
plt.figure(figsize=(10,4))
plt.title("Google's Stock Price")
plt.xlabel("Days")
plt.ylabel("Closed Price USD")
plt.plot(alpha['Close'])
plt.show()
```

Interpretation:



```
# To get close price
alpha=alpha[['Close']]
print(alpha)
```

Interpretation:

	Close
0	62.442001
1	63.227501
2	62.799999
3	63.172501
4	63.609001
...	...
1255	156.000000
1256	156.880005
1257	157.460007
1258	155.720001

1259 157.949997

[1260 rows x 1 columns]

```
# Creating variable for 'X'Days in future

futuredays=30

# To create a new unit X for units/day
alpha['prediction']=alpha[['Close']].shift(-futuredays)
print(alpha.head())
print(alpha.tail())
```

Interpretation:

Close prediction

0	62.442001	52.652500
1	63.227501	52.111000
2	62.799999	52.216999
3	63.172501	53.301998
4	63.609001	54.019001

Close prediction

1255	156.000000	NaN
1256	156.880005	NaN
1257	157.460007	NaN
1258	155.720001	NaN
1259	157.949997	NaN

```
# To create a data set to convert into numpy array
import numpy as np
x = np.array(alpha.drop(['prediction'], axis=1))[:-futuredays]
print(x)
y=np.array(alpha['prediction'])[:-futuredays]
```

```
print(y)
```

Interpretation:

```
[[ 62.442001]
 [ 63.227501]
 [ 62.799999]
 ...
 [132.559998]
 [135.240005]
 [136.289993]]
[ 52.6525  52.111  52.216999 ... 157.460007 155.720001 157.949997]
[[134.199997]
 [133.779999]
 [132.559998]
 [135.240005]
 [136.289993]]
```

```
# To divide training and testing data
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(x,y,test_size=0.25)
```

```
# To create a model
```

```
from sklearn.tree import DecisionTreeRegressor
tree=DecisionTreeRegressor().fit(xtrain, ytrain)
```

```
from sklearn.linear_model import LinearRegression
linear=LinearRegression().fit(xtrain,ytrain)
```

```
xfuture = alpha.drop(['prediction'],axis=1)[: -futuredays]
xfuture=xfuture.tail()
xfuture=np.array(xfuture)
print(xfuture)
```

```

# Decision tree prediction
treeprediction=tree.predict(xfuture)
print("Decision tree prediction:",treeprediction)

# Linear prediction
linearprediction=linear.predict(xfuture)
print("Linear prediction:",linearprediction)

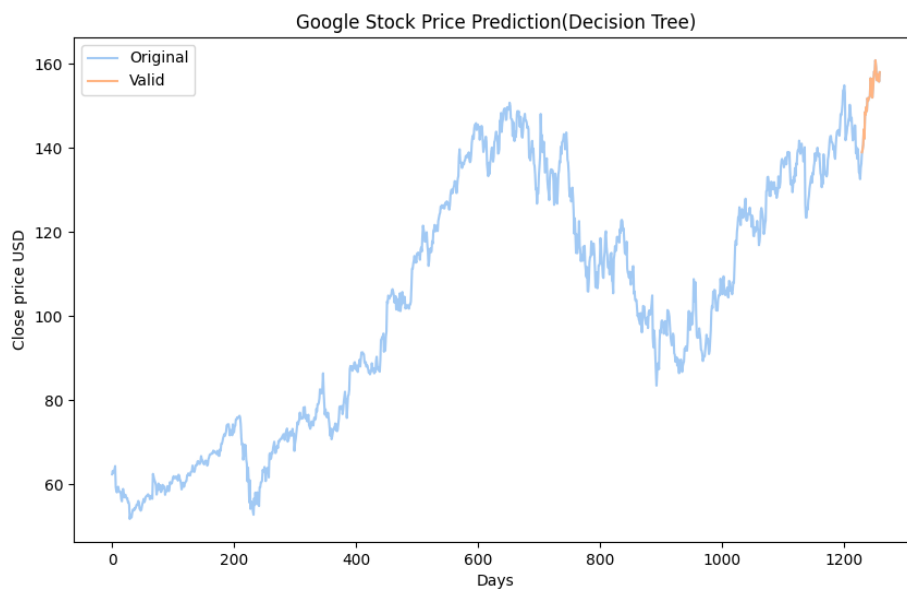
# To visualize decision tree prediction

prediction = treeprediction
valid=alpha[x.shape[0]:]
plt.figure(figsize=(10,6))
plt.title("Google Stock Price Prediction(Decision Tree)")
plt.xlabel('Days')
plt.ylabel('Close price USD')
plt.plot(alpha['Close'])
plt.plot(valid[['Close']])
plt.legend(["Original","Valid","Prediction"])
plt.show()

```

Interpretation:

Decision tree prediction: [128.374496 156.880005 157.460007 143.464996 145.205994]



Explanation:

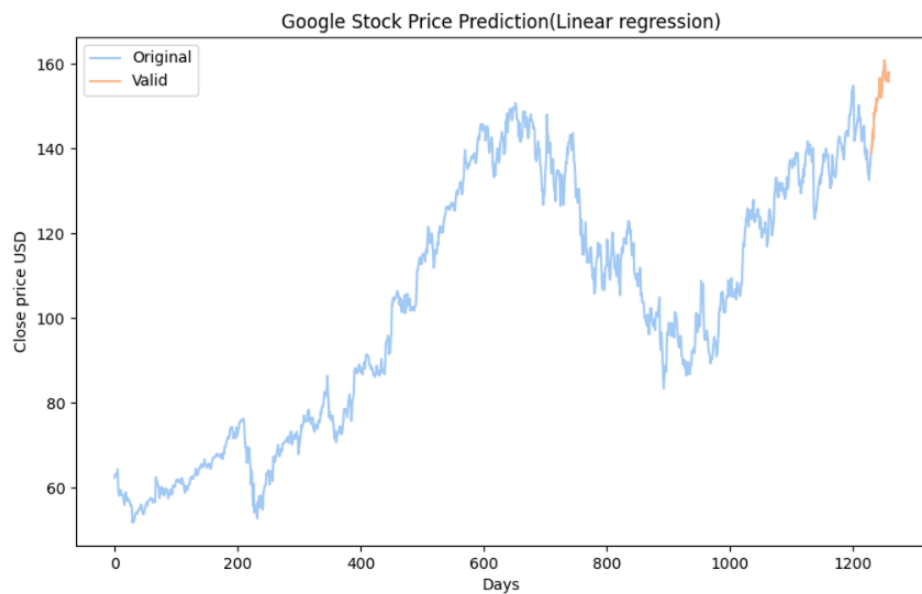
Line in orange is the predicted price for Google stock.

```
# To visualize linear prediction

prediction=linearprediction
valid=alpha[x.shape[0]:]
plt.figure(figsize=(10,6))
plt.title("Google Stock Price Prediction(Linear regression)")
plt.xlabel('Days')
plt.ylabel('Close price USD')
plt.plot(alpha['Close'])
plt.plot(valid[['Close']])
plt.legend(["Original", "Valid", "Prediction"])
plt.show()
```

Interpretation:

Linear prediction: [134.93592808 134.53594793 133.37409434 135.92636746 136.92631117]



Explanation:

The line in orange is the predicted price for Google stock.

```
# To validate the training model

from sklearn.metrics import mean_absolute_error

# To Calculate MAE for Decision Tree model
tree_mae = mean_absolute_error(ytest[:len(treeprediction)], treeprediction)
print("Mean Absolute Error (Decision Tree):", tree_mae)

# To Calculate MAE for Linear Regression model
linear_mae = mean_absolute_error(ytest[:len(linearprediction)], linearprediction)
print("Mean Absolute Error (Linear Regression):", linear_mae)
```

interpretation

Mean Absolute Error (Decision Tree): 33.6912018

Mean Absolute Error (Linear Regression): 23.241132295784777

Explanation: **Mean Absolute Error (MAE)** measures the average absolute errors between predicted values and actual values. The Linear Regression model has a lower MAE, indicating that, on average, its predictions are closer to the actual values than the Decision Tree model.

```
import numpy as np

# To Calculate squared errors
tree_squared_error = np.mean((ytest[:len(treeprediction)] - treeprediction) ** 2)
linear_squared_error = np.mean((ytest[:len(linearprediction)] - linearprediction)
** 2)

# To Calculate RMSE
tree_rmse = np.sqrt(tree_squared_error)
linear_rmse = np.sqrt(linear_squared_error)

print("Root Mean Squared Error (Decision Tree):", tree_rmse)
print("Root Mean Squared Error (Linear Regression):", linear_rmse)
```

Interpretation:

Root Mean Squared Error (Decision Tree): 43.72540924566654

Root Mean Squared Error (Linear Regression): 35.270735123683814

Explanation: **Root Mean Squared Error (RMSE)** measures the square root of the average of the squared differences between predicted values and actual values. The linear regression model has a lower RMSE, indicating that, on average, its predictions have smaller errors than the Decision Tree model.

Conclusion:

As per the prediction, the maximum price of Google stock reached nearly 157, and the lowest was 128.